

Contents

Future Ideas — Completed & Obsolete 1

- PlayVsBot start-flow — full collapse landed 2026-05-16 (filed 2026-05-13) 1
- Journey CTA spotlight wiring — resolved 2026-05-16 3
- Guide Help Subsystem (Chat Interface) — shipped 2026-05-15 (Help_System Sprints 1–4) 3
- Tournament admin UX overhaul — shipped 2026-05-10 3
- Bot Challenge & Discovery — shipped 2026-05-10 4
- Bot best-of-N series cap mis-formula — fixed 2026-05-09 5
- Pending PVP match registry per-process race — fixed 2026-05-09 5
- Recurring tournaments — template/occurrence schema refactor — shipped (Phase 3.7a) 6
- Tournament bot matches stuck IN_PROGRESS — fixed 2026-05-05 6
- Bot model half-converted state after train-guided — fixed 2026-05-05 6
- E2E journey suite cross-origin auth — fixed 2026-05-05 6
- Downstream journey-suite issues (post-cross-origin) — fixed 2026-05-05 7
- Real-Time Games Against Bots (e.g. Pong) — shipped Phase 6 7
- Real-Time Presence / Inactivity Detection — mostly done 7
- Multi-Game Architecture — largely done (as the Game SDK) 8
- Persist Game State Through Deploys (Redis-backed Rooms) — obsolete 9
- Configurable Guide — obsolete (premise gone) 9

Future Ideas — Completed & Obsolete

Companion doc to **Future_Ideas.md**. This file holds entries that were once “future ideas” or open bugs but have since been **shipped**, **resolved**, **superseded**, or **rendered obsolete**. Kept for archaeology — they often explain *why* the current architecture looks the way it does. Newest at top.

The active backlog (open and partly-shipped items only) lives in `Future_Ideas.md`.

□ PlayVsBot start-flow — full collapse landed 2026-05-16 (filed 2026-05-13)

All four originally-filed CTA items shipped. The 3-RTT start-flow chain is now collapsed into a single `POST /api/v1/play/bot`: the server pre-allocates the SSE session id and the client claims it via `?sseSession=<id>` when the shared `EventSource` opens. The opening board is returned in the same response, so the SSE round-trip is no longer on the perf-ready critical path.

Final shipped form (prod v1.4.0-alpha-5.7, 2026-05-16):

```
0ms  navigate
...  HTML + lazy JS chunk
~    POST /api/v1/play/bot → { sseSessionId, tableId, slug, board, mark, currentTurn, bot }
    setPlayBundle → applyCreateResultHvb → setPhase('playing')
~    board cell rendered (tReady)
...  in parallel ...
    EventSource open with ?sseSession=<id> (claims pre-allocated session via attachRes)
```

Structural collapse confirmed by `perf/perf-playvsvbot.js` (asserts run as part of the benchmark, not just metric tracking):

- `POST /api/v1/play/bot` — exactly **1** per run □
- `POST /api/v1/rt/tables` — **0** per run □ (collapsed)
- `POST /api/v1/rt/tables/:slug/join` — **0** per run □
- `GET /api/token` — **0** on critical path □
- `GET /api/v1/bots?gameId=xo` — **0** on critical path □

Mobile confidence — `perf/perf-playvsvbot-mobile.js` (Fast 3G profile via `CDP Network.emulateNetworkConditions` + Pixel 5 device): same harness, throttled to 1.6 Mbps down / 750 Kbps up / 150 ms RTT. Wired into `npm run qa` under “PlayVsBot mobile (Fast 3G)” entries for local / staging / prod.

Date	Version	Item	Status
2026-05-15	v5.5	1 — HvB join-chain trim	shipped
2026-05-15	v5.6	2 — [data-perf-ready] marker	shipped

Date	Version	Item	Status
2026-05-15	v5.6	3 — server /bots?gameId=cache	shipped
2026-05-15	v5.6	4 — eager initial-state event	confirmed fictional (struck)
2026-05-16	v5.7	3' — full POST /play/bot collapse	shipped
2026-05-16	v5.8	5 — /api/session shared-singleton dedup	shipped (8 → 2 GETs per cold mount; wall-clock unchanged at p50 because HTTP/2-multiplexed)

What shipped in v5.7:

- **backend/src/routes/play.js** — new POST /api/v1/play/bot. Thin orchestrator: resolves the community bot (or accepts caller-supplied botUserId), mints + pre-registers an SSE session id with res:null, calls tableFlow.createHvbTable, tracks the table on the pending session, returns { sseSessionId, tableId, slug, label, mark, board, currentTurn, bot }. Anonymous-OK. 12 vitest cases in play.test.js.
- **backend/src/realtime/sseSessions.js** — new attachRes(sessionId, { res, onDispose, userId }) helper. Upgrades a pending pre-allocated session by attaching the live SSE response + dispose callback. Preserves any tables already joined. 5 new vitest cases.
- **backend/src/routes/events.js** — /api/v1/events/stream now honors ?sseSession=<id>. If the id matches a pending pre-allocation it claims via attachRes; otherwise falls through to the existing mint-on-open behavior. Backward-compatible for every other flow (PvP, tournaments, demo, gym, admin).
- **landing/src/lib/useEventStream.js** — new claimSseSession(id) export stages a pre-allocated session id for the next openStream() call. One-shot; cleared after use.
- **landing/src/lib/useGameSDK.js** — accepts a playBundle parameter. When provided, skips waitForSseSession entirely: stages the bundle's session id, applies the create result synchronously, sets the tableId. The shared EventSource opens in parallel; the multi-step path is intact for PvP / joinSlug / tournaments / demo.
- **landing/src/pages/PlayPage.jsx** — vs-community-bot action now calls api.play.startBot() instead of getCommunityBot(), passes the response to <GameView playBundle={...}>. Module-level in-flight dedup so React StrictMode dev double-mount doesn't create a stranded duplicate table.
- **landing/src/lib/api.js** — api.play.startBot({ gameId, botUserId? }).
- **perf/perf-playvsbot.js** — extended assertions: tracks tokenGETs, botsListGETs, tablesCreatePOSTs, playBotPOSTs. Summary section calls out the structural collapse (one POST replaces three).
- **perf/perf-playvsbot-mobile.js** — new throttled-network mobile benchmark. Wired into npm run qa Perf menu (local / staging / prod).
- **e2e/tests/bot-challenge-flow.spec.js** — new test asserts HomePage “Play against a bot” CTA emits exactly 1 POST /play/bot and zero /rt/tables POSTs. Defense-in-depth regression guard against accidental fallback.
- **useGameSDK.sse.test.jsx** — 3 new cases pinning the single-shot bundle branch + the multi-step fallback for non-HvB flows.

What shipped in v5.8 (/api/session dedup, item 5):

- **landing/src/lib/useOptimisticSession.js** rewritten as a module-level singleton: one fetch + one 60-second poller regardless of subscriber count. The previous version fired one fetch per useOptimisticSession() instance on mount; 8 simultaneous callers (AppLayout, PlayPage, HomePage, JourneyCard, ProfileMenu, ...) hammered /api/session in parallel. New _listeners Set fans the shared state out to every subscriber; first mount starts polling, last unmount stops it. 6 vitest cases pin: (a) 5 parallel cold mounts → exactly 1 fetch; (b) shared state propagation; (c) triggerSessionRefresh() fires one fetch and notifies all subscribers; (d) failure → null; (e) anon → null without leaking the response shape; (f) last-unmount stops the poll.
- **Measured outcome:** /api/session GETs per cold mount went from **8** → **2** on both desktop and mobile staging benchmarks. tReady p50 was **unchanged** (~849 ms) because the pre-fix GETs were HTTP/2-multiplexed and never the longest pole at p50. What the dedup saved is real but invisible to perf-playvsbot: 6 redundant Better Auth session checks + console-log lines + tail-risk of one slow parallel GET pulling the median up.

Where the floor stands now (~842 ms desktop / 847 ms mobile-throttled prod warm-anon): the start-flow chain is structurally collapsed. Further wins require touching the cold-cache HTML/JS load path (lazy chunk splitting, SSR the opening board, render-before-auth-resolves) — see the “Larger items (deferred, for reference)” subsection in the active doc's tReady entry for the menu of options.

□ Journey CTA spotlight wiring — resolved 2026-05-16

Problem (as originally filed): the reusable `<Spotlight target={ref} active={...} onDismiss={...} />` component shipped on 2026-04-29 (landing/src/components/guide/Spotlight.jsx) and replaced the ad-hoc per-page `xo-spotlight-pulse` toggle. Steps 4 and 5 of the Curriculum journey were wired immediately (BotProfilePage Train + Spar buttons). Steps 3, 6, and 7 were filed as “wiring leftovers” — each estimated at ~30 minutes of `<Spotlight>` render-line + small destination handler.

What changed: on review, **none of the three remaining steps want a spotlight at all** — each has a better-fitting attention pattern that’s already in place:

- **Step 3 (?action=quick-bot)** — QuickBotWizard opens as a focused modal overlay (ProfilePage.jsx:116–121). A spotlight inside a modal adds no value — the modal already commands attention. **Closed: deliberately not wired.**
- **Step 4 (?action=train-bot)** — BotProfilePage Train button. □ wired
- **Step 5 (?action=spar)** — BotProfilePage Spar block; ProfilePage forwards the action to the bot detail page same way it does for train-bot. □ wired
- **Step 6 (?action=cup)** — handler now exists at ProfilePage.jsx:142–172. Calls `tournamentApi.cloneCurriculumCup`, spawns a 4-bot Curriculum Cup, registers the user’s bot, and immediately navigates to `/tournaments/<id>?follow=<botId>`. The `?follow=` query auto-opens the live spectate modal on arrival, which is the equivalent of a spotlight — actively pulls the user’s attention to their bot’s match. **Closed: spotlight pattern not applicable — user is forwarded away from /profile in a single tick; the follow-spectate is the destination attention-grab.**
- **Step 7 (?action=cup-result)** — JourneyCard.jsx:208–216 deliberately removed the clickable CTA. Step 7 fires server-side after the cup wraps; the CoachingCard auto-appears on completion (§5.5). The journey card renders an inline note in its place: “□ *Watching your cup play out — your result lands here automatically when it wraps.*” **Closed: replaced by auto-coaching-card surface, no CTA needed.**

Net: the `<Spotlight>` component has reached its final wiring footprint. Steps 3 / 6 / 7 each have a better attention pattern (modal overlay, follow-spectate, auto-coaching-card) than a spotlight would provide, so adding more wiring would be redundant.

□ Guide Help Subsystem (Chat Interface) — shipped 2026-05-15 (Help_System Sprints 1–4)

Problem (as originally filed): the “Ask Guide anything...” input at the bottom of the Guide panel was an unconnected placeholder. Wiring it up required deciding on context injection, conversation persistence, and cost controls.

What shipped: the full **Learnable Help System** — corpus + retrieval + admin editor + OpenAI-backed Q&A + browse pages + feedback + curation + metrics — over four sprints culminating in `v1.4.0-alpha-5.4` on prod (2026-05-15). The placeholder div in `GuidePanel.jsx` is now a real `HelpInput` + streamed `HelpAnswer` thread. The originally-proposed `POST /api/guide/chat` endpoint became `POST /api/v1/help/ask` (SSE-streamed, grounded in the corpus, not a free-form chat). Source of truth: `archive/Help_System_Sprint_Tracker.md` (Sprints 1–4 closed) + `archive/Help_System_Plan.md`.

Key deltas from the original sketch: - **Architecture:** the actual system is retrieval-augmented (corpus of 43 versioned `HelpDoc` rows + `pgvector` + FTS hybrid retrieval) rather than free-form chat. Cheaper, deterministic, easier to curate. Conversation persistence was scoped *out* — each turn is independent and grounded in the docs. - **Cost controls:** in-process per-user rate limiter (5/min, 50/day) in `helpRateLimit.js`. OpenAI project `aiarena` has a \$100/mo hard cap + \$10/\$25 alerts, \$5/mo on the local-dev key. - **Surfaces beyond the chat input:** Sprint 3 also shipped public `/help` + `/help/:slug` browse pages, and Sprint 4 shipped the admin curation queue + metrics dashboard so the corpus can be edited and grown from real traffic feedback. - **Post-launch follow-ups** live in the “**Help System — post-launch enhancements (Sprint 5)**” section of `Future_Ideas.md` (reranker v2, embedding upgrade, streaming filter, gap-filling from prod data, etc.).

□ Tournament admin UX overhaul — shipped 2026-05-10

Problem: Phase 3.7a separated `TournamentTemplate` from `Tournament` in the schema, but the operator-facing UI never caught up — the create/edit/manage flow for recurring tournaments was clumsy across nine specific dimensions catalogued in the original analysis (preserved at the bottom of this entry for archaeology).

What shipped (foundation sprint then final sprint):

- **Foundation sprint** (`h` → `i` → `c`): bare `<input type="date">` and `datetime-local` were swapped for the shared `<DateTimePicker>` (Safari-safe) plus a `<LocalTZ>` line showing “14:00 EDT / 18:00 UTC”; `TournamentForm` reorganised into three top-level groups — Basics / Schedule / Advanced — with recurrence promoted to a first-class subsection under Schedule; `TournamentModal` detects an occurrence’s `templateId` and renders an amber banner linking to the template editor so admins know edits there are local to *this* run.

- **Schedule preview** (b): pure helper landing/src/lib/recurrence.js#nextOccurrences mirrors the scheduler's advanceOne math. New "Schedule preview" panel on /admin/templates/:id renders the next 5 spawn times honoring recurrenceEndDate and paused (paused → "no occurrences" message).
- **Row-level series actions** (d): templates list ActionMenu now offers Pause/Resume series, **Stop series** (sets recurrenceEndDate=now() + paused=true so the scheduler permanently halts; reversible via clearing the end date but distinct from Delete), and **Clone** (deep-copies template config + seed bots, drops subscriptions, opens the clone in edit mode). New backend routes: POST /admin/templates/:id/stop and POST /admin/templates/:id/clone.
- **Embedded standing-registrations panel** (e): replaces the orphaned "look up by Template ID" form on AdminTournamentsPage with a "Standing registrations" panel directly on the template detail page. Shows display-named subscribers with missed-counts, opt-out toggle, an "Add subscriber" input that accepts username or userId, and an "Include opted-out users" checkbox for audit. New tournament-service routes: POST /api/recurring/:templateId/registrations (admin enrol by username/userId, idempotent), DELETE /api/recurring/:templateId/registrations/:userId (sets optedOutAt, preserves history), and ?includeOptedOut=true query on the existing list endpoint.
- **Clone-from on tournament rows** (g): AdminTournamentsPage row ActionMenu gained a Clone item that opens TournamentModal in a new mode='clone' — TournamentForm is prefilled from the source row with name suffix "(Copy)" and startTime / registrationOpenAt / registrationCloseAt cleared, falling through to tournamentApi.create() (not update).

What did not ship: items (a) and (f) were already present from earlier work — AdminTemplatesPage exists at /admin/templates and SeedBotsPanel is embedded on the template detail page.

Original analysis (kept for archaeology)

The schema refactor (Phase 3.7a — TournamentTemplate + templateId) shipped: see the "Recurring tournaments — template/occurrence schema refactor" entry below. What did **not** ship is the operator-facing UX that takes advantage of the cleaner model. Today the create/edit/manage flow for recurring tournaments is clumsy in nine concrete ways, all of which an admin runs into within their first few minutes of use.

Symptoms of the clumsiness (observed on prod 2026-05-09):

1. **One form does both jobs.** landing/src/components/tournament/TournamentForm.jsx (431 lines) creates a one-off tournament *and* a recurring template by toggling isRecurring. Recurrence config (interval, end date, pause, opt-out) is buried in a sub-section near the bottom of a single-column form.
2. **No schedule preview.** After creating a weekly template, nothing shows "next runs: Wed Apr 22, Wed Apr 29, Wed May 6 ...". Admins can't sanity-check the schedule without waiting for the sweep to spawn the next occurrence.
3. **Looking up a recurring series is broken.** RecurringRegistrations (AdminTournamentsPage.jsx:664) requires the admin to **paste a Template ID** into a text input. There's no clickthrough from the tournaments list to its registrations.
4. **Pause / Stop / Cancel-this-occurrence semantics aren't surfaced.** recurrencePaused is a checkbox inside the edit modal. There's no row-level "Pause series" / "Stop series" action; admins conflate cancelling one occurrence with stopping the whole thing.
5. **No edit-template-vs-edit-occurrence branching in the UI.** The schema separates them, but the form doesn't visually distinguish "you are editing the template (affects all future runs)" from "you are editing this occurrence."
6. **Seed bots and standing human registrations are managed in a separate panel.** Most "I want a weekly cup with these 4 bots" workflows take 3 surfaces — create form, seed-bots tab, registrations panel — when they should be one.
7. **No clone affordance.** Common operator action ("make another one like this on a different day") requires retyping every field.
8. **Date input inconsistency.** startTime uses the custom DateTimePicker (Safari-safe); recurrenceEndDate is a bare <input type="date"> with the just-shipped "no end date" toggle as a workaround. The fix is to drop bare native date inputs entirely and use the same picker.
9. **No timezone affordance.** Datetimes are entered in browser-local time with no display of the resolved UTC or "tournament starts 14:00 UTC for users in TZ X."

Plus a structural near-miss: isTest and isRecurring are 150 lines apart in the form — creating a test recurring series for QA is undocumented.

The shipped sprint above (foundation → final) addressed (h) → (i) → (c) → (b) → (d) → (e) → (g). Items (a) and (f) were already present.

□ Bot Challenge & Discovery — shipped 2026-05-10

Problem: "play another player's bot" was the platform's stated key differentiator but had no UX path. POST /rt/tables { kind: 'hvb', botUserId } and GET /api/v1/bots already supported it server-side; the gap was discovery + a one-click action surface.

What shipped (Phases A → C):

- **Primitives** (Phase A): `useBots()` SWR hook over the public bot list, `<BotCard>`, `<ChallengeButton>` (default + icon variants, guest-friendly, posts `/rt/tables` and navigates to `/play?join=<slug>`), `<BotFilterBar>` (search, ELO range, owner toggle, game select, “show all bots” toggle).
- **Surfaces** (Phase B): new `/bots` directory (scrollable `ListTable` of `Bot/Owner/ELO/Games/Play`); “Challenge” sections on `BotProfilePage` and the `/rankings` bot rows; “Challenge any bot” link on Home below “Watch another match”; About moved out of primary nav into the footer, replaced by Bots.
- **Pickers** (Phase C): tabbed bot picker in the Create Table modal (My / Community / All); Quick Match button on the directory header — calls new `GET /api/v1/bots/quick-match` (guest-OK, ELO-window candidate picker that excludes own bots).
- **Skill-gating**: backend `listBots()` now returns `playableGameIds` per bot, sourced strictly from `BotSkill` rows (legacy `botModelType='minimax'` bots without a skill row were false-positives — they reject at table-create with `NO_SKILL`). The directory filters to playable bots by default; a “Show all bots” toggle reveals the rest with a greyed-out Challenge button and a “No skill for XO” hint.
- **Context-aware navigation**: every linker to `/bots/:id` (directory rows, Rankings, `ProfilePage`’s “My bots”) passes `state.from`; `BotProfilePage`’s back link prefers state with a `/bots` fallback. `ChallengeButton/QuickMatchButton` thread the same state into `/play`; `PlayPage.leaveHref` honors it so “Leave Table” returns the user to the directory they came from instead of a hardcoded `/tables`.
- **Guest path**: every challenge surface works for unauthenticated visitors; the existing post-game signup CTA fires after the first finished game, not as a gate.
- **Tests**: unit coverage on each primitive + page + the quick-match route; new `e2e/tests/bot-challenge-flow.spec.js` covers the guest path through directory, profile, Quick Match, and Rankings.

Original plan doc: `Bot_Challenge_Plan.md` (deleted on completion — this entry is the digest).

❑ Bot best-of-N series cap mis-formula — fixed 2026-05-09

Symptom: every prod tournament with bot-vs-bot matches came in 10–35% over its theoretical bracket-game ceiling. Cora 3 (6-team SINGLE_ELIM bestOf3, all bots) played 20 games against an expected 15 — three of five played matches recorded `p1Wins=0`, `p2Wins=0` despite COMPLETED status, and showed exactly 5 games each (all draws + deterministic-tiebreaker resolution).

Root cause: `backend/src/realtime/botGameRunner.js:230` had `const hardCap = Math.max(bestOfN * 2 - 1, 1)`. The $2N - 1$ formula is correct for the *other* bestOf convention (where N is the wins required and $2N - 1$ is the max games — e.g. “best of 3 wins” = max 5 games). But this codebase uses `bestOfN` to mean **max games** (`winsNeeded = [N/2]` a few lines above), so the cap should just be N . For bestOf3 the cap was 5 games instead of 3. Two minimax bots playing optimally always draw → series ran the full 5-draw stretch every time before the deterministic tiebreaker resolved it.

Fix: extracted `seriesGameCap(bestOfN) = max(bestOfN, 1)` and used it. Existing tests only covered `bestOfN: 1`, so the broken path was never exercised. Added 4 new unit-test assertions covering `bestOf1/3/5 + nullish-fallback`. Bot-vs-bot tournaments now run noticeably faster and tournament game counts match the bracket math used by `expectedGameCount` and the runaway-loop guard.

❑ Pending PVP match registry per-process race — fixed 2026-05-09

Symptom: in a MIXED tournament with two human players paired in a round-1 HvH match, the second human’s “Play Match” click could return “Table closed due to inactivity” — surfacing as `useGameSDK setAbandoned({reason: 'stale'})` from a 404 on `POST /api/v1/rt/tournaments/matches/:id/table`. Single-machine staging never reproduced; prod intermittently did.

Root cause: `_pendingPvpMatches` in `backend/src/lib/tournamentBridge.js` was a per-process in-memory Map, populated from a Redis **pub/sub** subscription (not a stream — pub/sub does not replay missed messages). On multi-machine prod, three failure modes diverged the maps:

1. **Autoscale-up** — a new backend machine spawned after `tournament:match:ready` was published never received the event; its map stayed empty.
2. **Rolling deploy** — a redeployed machine lost its in-memory map; existing tournaments lost their entries on the restarted node.
3. **Transient Redis disconnect** — any subscriber gap during the publish missed the event.

`/rt/*` POSTs are routed via Fly-Replay to the SSE-owning machine. If user-1 and user-2’s SSE sessions landed on different machines, and user-2’s machine didn’t have the entry, the claim returned `NOT_FOUND` → 404.

Fix: replaced the in-memory Map with a Redis hash `tournament:pending:<matchId>` (TTL via `EXPIRE`, no JS-side prune needed). Falls back to in-memory when `REDIS_URL` is unset (tests / local-only). All four operations became async; six caller sites await them. Added 9 unit tests covering `get/set/delete/count` round-trips, slug-only updates, null-clear-on-disconnect, missing-key no-op, and `bestOfN` type coercion.

Verified on 2-machine staging: pre-fix the same shape caught 2/10 failures during a new-machine warmup window. Post-fix: 25/25 across 25 runs on a fresh 2-machine deploy. The race window structurally closes — there is no per-process state to diverge.

❑ Recurring tournaments — template/occurrence schema refactor — shipped (Phase 3.7a)

Original problem: a recurring tournament was modelled as a single Tournament row with `isRecurring: true` that *also ran as the first occurrence*. The template was both a configuration row *and* a historical record of the first run, and “what recurring series exist?” couldn’t be answered without filtering. Admins also conflated “cancel this occurrence” with “stop the whole series.”

What shipped (Phase 3.7a):

- New TournamentTemplate table holds the recurrence config (interval, end date, paused, seed bots, human subscriptions). It never itself runs.
- Every Tournament row is now a pure occurrence with `templateId?: string`. The schema removed `isRecurring / recurrenceInterval / recurrencePaused` from Tournament.
- New TournamentTemplateSeedBot and TournamentRecurringRegistration tables key on `templateId`, not the first occurrence’s tournament id.
- Admin surfaces shipped: `landing/src/pages/admin/AdminTemplatesPage.jsx + AdminTemplateDetailPage.jsx`. The recurring scheduler in the tournament service was rewritten to advance from the template, not the first row.

What this entry left behind for the operator UX: see the “Tournament admin UX overhaul” entry above. The schema landed cleanly in Phase 3.7a; the operator-facing surfaces around it (schedule preview, embedded seed/registration mgmt, edit-template-vs-occurrence branching, clone, TZ display, ...) shipped in the follow-up sprint described above.

❑ Tournament bot matches stuck IN_PROGRESS — fixed 2026-05-05

Original symptom: every Curriculum Cup on staging stuck IN_PROGRESS with `updatedAt === createdAt` and `p1Wins=p2Wins=0`. Bot games actually completed in-memory but the bracket never advanced; journey step 7 never fired; 4 cup-soak e2e suites timed out at 240s.

Root cause: `TOURNAMENT_SERVICE_URL` secret on staging was set to `http://xo-tournament-staging.flycast:3001`. `.flycast` is Fly’s anycast private DNS, but only resolves when the target app has a private IPv6 IP allocated (`fly ips allocate-v6 --private`). Our deploy doesn’t allocate one, so DNS lookups returned `ENOTFOUND`, every cup match completion POST silently failed (errors logged at warn then swallowed), and the bracket-advance pipeline never ran. Prod was using `https://xo-tournament-prod.fly.dev` (public hop) so it worked but at extra latency.

Fix: flip both backend services to `http://xo-<env>-tournament.internal:3001` (Fly 6PN private DNS — resolves without any IP allocation, same private network as `.flycast` would have provided). Verified: a clean cup against staging now completes in ~1 minute and step 7 credits. Runbook updated (`Prod_Bringup_Runbook.md` §xo-backend-prod) so this doesn’t recur.

❑ Bot model half-converted state after train-guided — fixed 2026-05-05

Symptom: after `POST /api/v1/bots/:id/train-guided/finalize`, the bot ended up with `botModelType: 'minimax'` (unchanged) but `botModelId: <BotSkill UUID>` — half-way between Quick Bot and trained ML bot. Caused journey step-4 e2e tests (`guide-onboarding`, `guide-ui-states`, `journey-train-modal`) to fail asserting `botModelType === 'qlearning'`.

Root cause: `mlService.js#repointBotPrimarySkill` (and the duplicate in `skillService.js`) updated `User.botModelId` when training completed but didn’t touch `botModelType`. The finalize handler at `bots.js:469` then saw `bot.botModelId === skillId` (already aligned) and skipped the `qlearning` flip. Verified on staging — DB rows for recent train-guided bots showed exactly this state.

Fix: both `repointBotPrimarySkill` functions now read `algorithm` from the BotSkill row and write `botModelType` derived from it (lower-snake-no-underscore: `Q_LEARNING` → `qlearning`, `MONTE_CARLO` → `montecarlo`, etc.). Backwards-compatible — no `botModelType` write when `algorithm` is null. 7 new unit tests across `mlService` + `skillService`.

❑ E2E journey suite cross-origin auth — fixed 2026-05-05

Original symptom: `e2e/tests/guide-onboarding.spec.js` (and 8 sibling specs) failed immediately on staging/prod with `No token in response` — user may not be signed in on this context from `helpers.js:111`. Test signed up on `xo-landing-*.fly.dev`

(cookie on landing origin); fetchAuthToken then hit `${BACKEND_URL}/api/token` on `xo-backend-*.fly.dev` — different subdomain → no cookie → 401.

Fix: route every test API call through the landing host so the existing landing/server.js proxy (pathFilter: `['/api', '/socket.io']`) carries the cookie + forwards to backend. Replaced `BACKEND_URL` with `LANDING_URL` across all 9 affected specs (88 references). Backend CORS already allows the landing origin via `FRONTEND_URL`; no backend changes needed. The working pattern was already proven in `smoke.journey.spec.js`.

Files patched: `journey-cup · guide-hook · guide-onboarding · guide-ui-states · guide-curriculum · journey-spar · journey-train-modal · journey-spotlight · guide-phase0`.

□ Downstream journey-suite issues (post-cross-origin) — fixed 2026-05-05

After the cross-origin auth fix unblocked the suite, several smaller test/staging issues surfaced and were resolved in the same sprint:

- **guide-hook step 1 prefs 4xx** — GET `/api/v1/guide/preferences` 404'd because the BetterAuth → application User row hadn't been mirrored yet. Fix: explicit POST `/users/sync` after fetchAuthToken, matching the existing pattern in `guide-onboarding/journey-cup`.
- **guide-hook step 2 demo-watch credit timing** — same race; same `/users/sync` fix.
- **guide-hook private-table assertion** — test asserted demo Table rows must NOT appear in default GET `/api/v1/tables` for the creator, but backend's documented behavior is “authed users see their own private tables in default” (see `tables.js:276`). Replaced with the right privacy check: an anonymous viewer must not see the demo.
- **guide-onboarding step 4 model-type** — covered above (the `repointBotPrimarySkill` fix).
- **Stale slug regex** — `guide-hook.spec.js:80` and `guide-curriculum.spec.js:113` asserted `/^mt-/` but backend uses `nanoid(8)` (no prefix). Updated to `/^[A-Za-z0-9_-]{8}$/`.
- **Username collisions** — hardcoded display names (Hook Demo, Phase Zero, Curr Spar, etc.) collided on the lower-snake-cased username unique constraint after the first staging run. Added `uniqueName(label)` helpers that suffix a random tag.

After these fixes plus the cross-origin and `repointBotPrimarySkill` work, the full journey suite (25 tests across 9 specs) runs green end-to-end on staging.

□ Real-Time Games Against Bots (e.g. Pong) — shipped Phase 6

Status: real-time games shipped as Phase 6. `backend/src/realtime/pongRunner.js` runs a server-authoritative tick loop; client SDK lives in `landing/src/lib/usePongSDK.js`; POST routes are mounted under `/api/v1/rt/pong/rooms` (create / join / input). The XO turn-based game and the Pong real-time game now share the same Table row primitive but use different runners.

Original analysis (kept for archaeology — the design ahead of shipping):

- **Game loop:** server-authoritative tick loop (e.g. 60Hz) running in the backend.
 - **Bot model:** AI input is a continuous state vector (ball position, velocity, paddle positions) rather than a discrete board encoding. Suitable algorithms: DQN or Policy Gradient trained on the continuous state space.
 - **Rendering:** a canvas or WebGL game loop on the frontend replacing the current board grid.
 - **Recording:** game outcome (win/loss/score) POSTed to `/games` at completion — credit and ELO hooks unchanged.
 - **PvP extension:** two human players play real-time games against each other via the existing realtime infrastructure, with the server relaying inputs and authoritative state.
-

□ Real-Time Presence / Inactivity Detection — mostly done

Status: The core heartbeat-based presence was built during the Phase E tier-2 comms work. `backend/src/lib/presenceStore.js` tracks `onlineAt` with TTL; `landing/src/lib/useHeartbeat.js` is the client hook; GET `/api/v1/presence/online` + the `presence:changed` SSE channel expose the state.

What's still open (small refinement, not blocking): the hide/show behaviour uses default visibility — when a tab backgrounds, the heartbeat stops and the server evicts the user after the TTL. There's no explicit `user:away` transition and no distinction between “tab hidden” (may come back in seconds) and “tab closed” (likely gone for the day). Good enough for the admin “online” indicator in its current form.

❑ Multi-Game Architecture — largely done (as the Game SDK)

Status: The Game Adapter pattern proposed here was implemented as the GameSDK contract in Platform Phases 1.1–1.4. `packages/sdk` defines `GameMeta`, `GameSDK`, and `botInterface`; `@calliduity/game-xo` is the reference implementation; `PlatformShell` loads any `GameMeta`-conforming package via `React.lazy`. `roomManager` was retired in Phase 3.4 — `Table` rows + `previewState` + SDK adapters replaced it.

What's left (tracked in `doc/Platform_Implementation_Plan.md`): - **Phase 4** — Connect4 (2p sit-down, validates the abstraction with a second game) - **Phase 5** — Poker (adds hidden-info via `getPlayerState`, variable player counts) - **Phase 6** — Pong (adds `tableArchetype`: 'head-to-head' + real-time loop) — ❑ shipped (see above)

The original analysis below remains a useful reference for the per-game rendering strategy (React vs Framer Motion vs Phaser), but the high-level adapter/registry work is done.

Original analysis — what: Evolve the platform to support additional game types — Connect 4, Checkers, card games, and real-time games like Pong — without rewriting the infrastructure that already works.

What's already generic: The room lifecycle (`create/join/disconnect/reconnect/close`), socket event envelope (`game:start`, `game:moved`), ELO system, game recording, mountain name rooms, spectator system, and credits (`appId` already on the schema) are all game-agnostic today. They need no changes to support new games.

What's XO-hardcoded today: `board: Array(9).fill(null)`, `makeMove({ cellIndex })`, `getWinner/WIN_LINES/isBoardFull` called directly inside `roomManager` and `botGameRunner`, `playerMarks: X|O`, and `winLine`. `GameBoard.jsx` (~600 lines) and `gameStore.js` are XO-specific. The AI registry calls `aiImpl.move(board, difficulty, currentTurn)` — an XO-shaped interface.

The core abstraction: a Game Adapter

Each game type registers an adapter that owns its rules. The room manager and bot runner stop knowing anything about game logic and delegate to it:

```
{
  appId: 'connect4',
  initialState()                // returns a fresh game state
  applyMove(state, move)        // returns { state, terminal, winner }
  validateMove(state, move, playerMark) // boolean
  serializeForClient(state)      // what gets emitted over the socket
}
```

`roomManager.makeMove()` currently calls `getWinner(room.board)` directly. With adapters it becomes `adapter.applyMove(room.gameState, move)`. The room carries a `gameType` field and the manager looks up the adapter from a registry — the same pattern as the existing AI registry. On the frontend, `GameBoard.jsx` splits into a generic `GameContainer` (socket connection, room management, scores, forfeit, spectator mode) and a game-specific renderer (`XOBoard`, `Connect4Board`, etc.) that receives standardized state and emits standardized moves.

The four game types and what each requires:

- **Connect 4** — most similar to XO. Different board shape (7×6), gravity mechanic (move is a column index, not a cell index), 4-in-a-row win detection. Fits the adapter interface cleanly. Good first target for proving the abstraction.
- **Checkers** — still turn-based and discrete, but move validation is complex (forced captures, multi-jump chains, kinging). The adapter interface handles it — `validateMove` and `applyMove` just do more work. Socket model is unchanged because state is fully visible to both players.
- **Card games** — turn-based discrete moves, but with hidden information (hand cards). The current socket broadcast model breaks here: the server can't emit full state to the room because each player should only see their own hand. The adapter interface needs a `serializeForPlayer(state, playerMark)` method, and the socket layer must emit per-player views (`io.to(socketId).emit()`) instead of broadcasting to the room. This is the meaningful socket architecture change for card games.
- **Pong / real-time** — fundamentally different. No turns, no discrete moves. Needs a `RealtimeGameRunner` with a server-authoritative tick loop (or client-side simulation with the server recording the final result). The existing `BotGameRunner._runGameLoop` async loop is the conceptual ancestor but would need to run at ~60Hz and push continuous state.

Rendering strategy and Phaser:

No single renderer fits all game types:

Game	Renderer
XO	React (existing)
Connect 4	React + CSS transitions
Checkers	React + CSS transitions
Card games	React + Framer Motion
Pong / real-time	Phaser (lazy-loaded)

Phaser is a complete 2D game framework (WebGL/canvas, physics engine, 60Hz game loop, sprite management, input handling). It operates outside React’s DOM model — you mount it imperatively in a `useEffect` and tear it down on cleanup. It’s the right tool for real-time physics games (Pong) where it saves significant manual work on collision, velocity, and game loop management. It’s overkill for turn-based games.

PixiJS is a lighter alternative (~400KB vs Phaser’s ~1MB+): WebGL rendering without the physics engine. Better fit if a game needs smooth sprite rendering but not physics — certain card game animations, animated boards.

Critical: Phaser (or PixiJS) must be a **lazy-loaded, per-game dependency** — not a platform-wide import. A player loading Connect 4 should never download Phaser. The game adapter architecture supports this naturally: each game’s renderer is its own bundle chunk, loaded only when that game is selected.

Recommended evolution path:

1. **Phase 1 — Extract and prove the adapter (Connect 4):** Move XO logic out of `roomManager` and `botGameRunner` into `XOGameAdapter`. Create the `gameAdapters` registry. Add `Connect4GameAdapter` and `Connect4Board.jsx`. Split `GameBoard.jsx` into `GameContainer` + `XOBoard`. This validates the abstraction without breaking anything.
2. **Phase 2 — Checkers:** Adapter interface unchanged; more complex `validateMove/applyMove`. No socket changes.
3. **Phase 3 — Card games:** Add `serializeForPlayer` to the adapter interface. Add per-player socket emission to the socket layer.
4. **Phase 4 — Real-time (Pong):** Separate architecture path. `RealtimeGameRunner`, canvas renderer via Phaser, tick loop. Plan independently once at least one more turn-based game exists.

Complexity: Phase 1 is medium (~3–4 days — the adapter extraction plus a working Connect 4). Each subsequent phase builds on it. The real-time phase is large and largely independent.

❑ Persist Game State Through Deploys (Redis-backed Rooms) — obsolete

Status: Superseded by Phase 3.4 — `roomManager` was retired. Active games are now `Table` rows in Postgres (`previewState Json` + `seats Json`), so game state survives deploys by design. Socket reconnection after a brief drop re-joins the table room via the `TableDetailPage` flow. The scenario this item was guarding against no longer exists.

Residual concern worth tracking separately: when a socket drops mid-game, `useGameSDK` needs to rebind the move stream — today there’s a short window where a move could be emitted to a disconnected socket. This is a socket-reconnect concern, not a state-persistence one, and is better filed as a gameplay-robustness task if it ever surfaces.

❑ Configurable Guide — obsolete (premise gone)

Status: This item was written against the old iframe-based `public/getting-started.html` + 9-balloon SVG layout. That page no longer exists — it was retired during the Phase 2 nav restructure and the Phase 3.3 Guide panel rebuild. The Guide is now a React drawer (`landing/src/components/guide/GuidePanel.jsx`) with a journey card, slots grid, and notifications feed. Any future “configurable guide” work would be a fresh design against the new shell, not a continuation of this item. Leaving the original text below for archaeology:

What: Let users personalize the Getting Started guide through a “Configure Guide” panel in Settings. Three layers of configuration, in increasing complexity:

1. **Arrow toggle** — a switch to show or hide the dashed connector arrows between balloons. Some users find them helpful for understanding the progression; returning users who use the guide as a launcher find them visual noise.
2. **Balloon count** — a slider or stepper (1–9, the current maximum) controlling how many balloon positions are shown. Fewer balloons means a less cluttered guide focused on the actions the user actually uses. Hidden positions render empty — the layout

stays fixed so the guide doesn't reflow.

3. **Balloon assignment** — a drag-and-drop configurator where the user picks which function occupies each position. A palette lists all available destinations (Play, Gym, Puzzles, Rankings, Stats, Profile, About, FAQ, Settings, plus the Feedback and Have Fun easter eggs). The user drags a destination from the palette onto a slot in a miniature preview of the guide layout. The resulting assignment is saved and the guide renders accordingly.
4. **Presets** — 3–4 named configurations selectable with a single click, shown at the top of the Configure Guide panel before the manual controls. Selecting a preset populates the arrow toggle, balloon count, and slot assignments all at once; the user can then fine-tune from there. Candidate presets:
 - **Default** — the current fixed layout (all 9 balloons, arrows on, original assignments). Restores the out-of-the-box experience.
 - **Onboarding** — arrows on, all balloons visible, ordered as a learning path (FAQ → Play → Training Guide → Create Bot → Train → Compete).
 - **Launcher** — arrows off, 5–6 balloons showing only the most-used destinations (Play, Gym, Leaderboard, Profile, Puzzles). Optimized for returning users who treat the guide as a quick-action menu.
 - **Minimal** — arrows off, 3 balloons (user-chosen or defaulting to Play, Gym, Profile). Maximum signal, minimum clutter.

Balloon actions beyond simple navigation:

Each balloon in the palette would be associated with an *action*, not just a URL. An action is a small descriptor like `{ to: '/profile', open: 'bots' }` or `{ to: '/gym', focus: 'model-name' }`. When the user clicks the balloon, the guide posts the action to the parent via `postMessage`; the parent closes the modal and calls React Router's `navigate(to, { state: action })`. The destination page reads `location.state` on mount and performs the side effect — opening an accordion, scrolling to a section, setting focus on an input, pre-selecting a tab, etc.

This means the palette of available destinations is really a palette of *actions*, each with a label, an emoji, a destination route, and an optional UI side effect.